

ESTRUTURA DE ARQUIVOS EM C — FOLHA DE COLA

0. Bibliotecas

```
#include <stdio.h> // FILE fopen fclose fgetc fputc fread fwrite
// fseek ftell rewind feof printf fprintf sprintf
#include <stdlib.h> // malloc free qsort
#include <string.h> // strcmp strncmp strcpy strncpy memcpy memset
```

Programa	Bibliotecas
copiar / contar (fgetc)	stdio
busca binária direta	stdio + string
ordenar / índice / ord. externa	stdio + stdlib + string

stdin teclado · stdout tela · stderr erro → fprintf(stderr, ...)

1. Abrir e fechar

```
FILE *f;
f = fopen(argv[1], "rb"); // FILE* ou NULL
if(!f){
    fprintf(stderr, "Arquivo %s não pode ser aberto para leitura\n",
        argv[1]);
    return 1;
}
fclose(f); // nunca fclose(*f)

"rb" lê (precisa existir) "wb" escreve (cria; APAGA)
"r+b" lê e escreve (existir) "a" escreve no fim
r=read w=write a=append +=lê e escreve b=binary
```

2º fopen falhou → fecha o 1º antes do return:

```
saida = fopen(argv[2], "wb");
if(!saida){
    fclose(entrada);
    fprintf(stderr, "Arquivo %s não pode ser aberto para escrita\n",
        argv[2]);
    return 1;
}
```

argc / argv (nome do programa conta):

```
if(argc != 2){ // programa + 1 arg (argc != 3 → origem + destino)
    fprintf(stderr, "USO: %s [CEP]\n", argv[0]);
    return 1;
}
```

2. Ler e escrever

1 byte: fgetc / fputc

```
int c = fgetc(f); // int: precisa caber o EOF
fputc(c, f); // byte primeiro, arquivo depois
```

Molde: ler → testar → usar → ler de novo

```
int c;
c = fgetc(entrada);
while(c != EOF){
    /* tarefa */
    c = fgetc(entrada);
}
```

Programa	/* tarefa */	Antes
Copiar	fputc(c, saida);	abrir entrada e saida
Contar linhas	if(c == '\n'){ count++; }	int count = 0;
Contar bytes	contador[c]++;	int contador[256] = {0};

Imprimir só letras:

```
for(int i = 0; i < 256; i++){
    if(contador[i] > 0 &&
        ((i >= 'A' && i <= 'Z') || (i >= 'a' && i <= 'z'))){
        printf("%c: %d\n", i, contador[i]);
    }
}
```

printf("%c",65) → A · printf("%d",65) → 65 · 1 byte = 8 bits = 256 valores

Vários itens: fread / fwrite

fread (ONDE, TAMANHO, QUANTOS, ARQUIVO)	ARQUIVO → MEMÓRIA
fwrite (ORIGEM, TAMANHO, QUANTOS, ARQUIVO)	MEMÓRIA → ARQUIVO
retorno = nº de ITENS (não bytes)	

```
fread(&e, sizeof(Endereco), 1, f); // struct → &
fread(e, sizeof(Endereco), qtd, f); // ponteiro/array → sem &
fwrite(e, sizeof(Endereco), qtd, saida);

if(fread(e, sizeof(Endereco), qtd, f) == qtd){
    printf("Lido = OK\n");
}
```

Copiar em blocos:

```
#define TAMANHO 8192
char buffer[TAMANHO];
int qtd;
qtd = fread(buffer, sizeof(char), TAMANHO, entrada);
while(qtd > 0){
    fwrite(buffer, sizeof(char), qtd, saida); // qtd, não TAMANHO
    qtd = fread(buffer, sizeof(char), TAMANHO, entrada);
}
```

3. Posição no arquivo

```
fseek(f, deslocamento_em_BYTES, origem); // MOVE
long pos = ftell(f); // INFORMA (bytes desde o início)
rewind(f); // VOLTA ao início
feof(f); // fim? (só depois de uma leitura falhar)
```

Origem	Conta a partir de	Exemplo → resultado
SEEK_SET	início	fseek(f,300,SEEK_SET) → byte 300
SEEK_CUR	posição atual	fseek(f,300,SEEK_CUR) → anda 300 fseek(f,-300,SEEK_CUR) → volta 300
SEEK_END	fim	fseek(f,0,SEEK_END) → vai pro fim

```
fseek(f, 0, SEEK_SET); // início (= rewind)
fseek(f, 0, SEEK_END); // fim → medir arquivo
fseek(f, meio * sizeof(Endereco), SEEK_SET); // vai ao registro meio
fseek(f, -sizeof(Endereco), SEEK_CUR); // volta 1 registro
```

```
fseek MOVE (não lê) ftell INFORMA (não move)
fread LÊ E AVANÇA (600 + 300 lidos → 900)
rewind VOLTA AO INÍCIO
```

Contas

```
// bytes → registros (medir)
fseek(f, 0, SEEK_END);
long tamanhoBytes = ftell(f);
long qtd = tamanhoBytes / sizeof(Endereco);
rewind(f); // se o próximo passo for fread

// registro → bytes
fseek(f, n * sizeof(Endereco), SEEK_SET);
// 499 × 300 = 149.700 → fseek(f, 149700, SEEK_SET)

// partes
long divisao = quantidade / PARTES;
long resto = quantidade % PARTES;
e = (Endereco*) malloc((divisao + 1) * sizeof(Endereco));
// 100 reg, 8 partes → div 12, resto 4 → partes 0-3: 13 · 4-7: 12
```

4. Memória

```
Endereco *e = (Endereco*) malloc(qtd * sizeof(Endereco));
if(e == NULL){
    fprintf(stderr, "Erro ao alocar memoria\n");
    fclose(f);
    return 1;
}
free(e);

memset(p, 0, 1000 * sizeof(Pessoa)); // (endereço, valor, BYTES)
Pessoa p[1000] = {0}; // zerar simples
```

sizeof(Endereco) = UM registro (300), não o arquivo · malloc reserva · memset preenche · free libera · **FILE* nunca leva malloc/free**

5. Strings (campos fixos, sem '\0')

```
strcpy(indice[i].cep, e.cep, 8); // copia 8
strcpy(p[0].nome, "Renato Mauro"); // array não usa =
sprintf(nome, "cep %d.dat", i); // i=3 → "cep 3.dat"
printf("%.8s\n", e.cep); // máx. 8 caracteres
int r = strcmp(a, b, 8);
// r < 0 a ANTES · r == 0 iguais · r > 0 a DEPOIS
```

6. Ponteiros

```
FILE *f; f → FILE* (endereço) *f → FILE
f = fopen OK *f = fopen ERRADO:
Endereco e; e → struct &e → endereço
Endereco *e; e → endereço *e → struct apontada
e.campo → struct e->campo → ponteiro (= (*e).campo)
e[i].campo → elemento i do bloco
```

Situação	Declaração	malloc	Campo	fread
1 por vez	Endereco e;	não	e.cep	&e
vários	Endereco *e;	sim	e->cep / e[i].cep	e
arquivo	FILE *f;	nunca	—	—

7. qsort e compara

```
qsort(vetor, quantidade, sizeof(Tipo), compara); // compara SEM ()
```

Função	2º	3º
fread / fwrite	TAMANHO	QUANTIDADE
qsort	QUANTIDADE	TAMANHO

sizeof do qsort e casting do compara = **tipo do bloco** ordenado.

```
// Endereco (ordenar arquivo / partes)
int compara(const void *e1, const void *e2){
    return strcmp(((Endereco*)e1)->cep, ((Endereco*)e2)->cep, 8);
}
qsort(e, qtd, sizeof(Endereco), compara);

// IndiceCep (índice)
int compara(const void *a, const void *b){
    return strcmp(((IndiceCep*)a)->cep, ((IndiceCep*)b)->cep, 8);
}
qsort(indice, qtd, sizeof(IndiceCep), compara);
```

8. Programas

Structs

```
typedef struct _Endereco Endereco;
struct _Endereco{
    char logradouro[72];
    char bairro[72];
    char cidade[72];
    char uf[72];
    char sigla[2];
    char cep[8];
    char lixo[2]; // espaço + \n → 300 bytes
};

struct indiceCep{
    char cep[8];
    long posicao; // nº do registro no ORIGINAL
};
typedef struct indiceCep IndiceCep;
```

8.1 Ordenar arquivo inteiro (fread → qsort → fwrite)

```
f = fopen("cep.dat", "rb");
fseek(f, 0, SEEK_END);
posicao = ftell(f);
qtd = posicao / sizeof(Endereco);
e = (Endereco*) malloc(qtd * sizeof(Endereco));
rewind(f);
if(fread(e, sizeof(Endereco), qtd, f) == qtd){
    printf("Lido = OK\n");
}
qsort(e, qtd, sizeof(Endereco), compara);
saida = fopen("cep-ordenado.dat", "wb");
fwrite(e, sizeof(Endereco), qtd, saida);
fclose(saida);
fclose(f);
free(e);
```

8.2 Busca binária direta (arquivo ORDENADO)

```
f = fopen("cep_ordenado.dat", "rb");
fseek(f, 0, SEEK_END);
long tamanhoBytes = ftell(f);
long tamanhoRegistros = tamanhoBytes / sizeof(Endereco);
long inicio = 0;
long fim = tamanhoRegistros - 1;

while(inicio <= fim){
    long meio = (inicio + fim) / 2;
    fseek(f, meio * sizeof(Endereco), SEEK_SET); // a cada tentativa
    fread(&e, sizeof(Endereco), 1, f);
    if(strncmp(argv[1], e.cep, 8) == 0){
        printf("%.72s\n%.72s\n%.72s\n%.72s\n%.2s\n%.8s\n",
            e.logradouro, e.bairro, e.cidade, e.uf, e.sigla, e.cep);
        break;
    }else if(strncmp(argv[1], e.cep, 8) > 0){
        inicio = meio + 1; // maior → direita
    }else{
        fim = meio - 1; // menor → esquerda
    }
}
fclose(f);
```

8.3 Índice (fseek só quando acha)

```
Endereco e; // 1 por vez
IndiceCep *indice; // todas as entradas

fseek(f, 0, SEEK_END);
tamanhoBytes = ftell(f);
qtd = tamanhoBytes / sizeof(Endereco);
indice = malloc(qtd * sizeof(IndiceCep));
rewind(f);

for(i = 0; i < qtd; i++){
    fread(&e, sizeof(Endereco), 1, f);
    strncpy(indice[i].cep, e.cep, 8);
    indice[i].posicao = i;
}
qsort(indice, qtd, sizeof(IndiceCep), compara);

long inicio = 0;
long fim = qtd - 1;
while(inicio <= fim){
    long meio = (inicio + fim) / 2;
    int resultado = strcmp(argv[1], indice[meio].cep, 8);
    if(resultado == 0){
        long posicao = indice[meio].posicao;
        fseek(f, posicao * sizeof(Endereco), SEEK_SET);
        fread(&e, sizeof(Endereco), 1, f);
        printf("%.72s\n%.72s\n%.72s\n%.72s\n%.2s\n%.8s\n",
            e.logradouro, e.bairro, e.cidade, e.uf, e.sigla, e.cep);
        break;
    }else if(resultado > 0){
        inicio = meio + 1;
    }else{
        fim = meio - 1;
    }
}
free(indice);
fclose(f);
```

Ordenar o índice **não muda posicao**: 22222222 → 0 continua apontando p/ o registro 0.

	Busca direta	Índice
Arquivo ordenado?	Sim	Não
Compara onde?	arquivo	memória
fseek	a cada tentativa	só quando acha

8.4 Ordenação externa — dividir e ordenar

```
#define PARTES 8

cep = fopen("cep.dat", "rb");
fseek(cep, 0, SEEK_END);
long tamanho = ftell(cep);
long quantidade = tamanho / sizeof(Endereco);
long divisao = quantidade / PARTES;
long resto = quantidade % PARTES;
rewind(cep);

e = (Endereco*) malloc((divisao + 1) * sizeof(Endereco));

for(int i = 0; i < PARTES; i++){
    long quantidadeParte = divisao;
    if(i < resto){
        quantidadeParte++;
    }
    fread(e, sizeof(Endereco), quantidadeParte, cep); // continua
    qsort(e, quantidadeParte, sizeof(Endereco), compara);
    sprintf(nome, "cep %d.dat", i);
    FILE *parte = fopen(nome, "wb");
    fwrite(e, sizeof(Endereco), quantidadeParte, parte);
    fclose(parte);
}
fclose(cep);
```

8.4 Intercalação (lê ANTES do while)

```
void intercala(char *arqA, char *arqB, char *arqSaida){
    FILE *a, *b, *saida;
    Endereco ea, eb;

    a = fopen(arqA, "rb");
    b = fopen(arqB, "rb");
    saida = fopen(arqSaida, "wb");

    if(!a || !b || !saida){
        fprintf(stderr, "Arquivo nao pode ser aberto.\n");
        return; // void → return sem valor
    }

    fread(&ea, sizeof(Endereco), 1, a);
    fread(&eb, sizeof(Endereco), 1, b);

    while(!feof(a) && !feof(b)){
        if(compara(&ea, &eb) < 0){
            fwrite(&ea, sizeof(Endereco), 1, saida);
            fread(&ea, sizeof(Endereco), 1, a);
        }else{
            fwrite(&eb, sizeof(Endereco), 1, saida);
            fread(&eb, sizeof(Endereco), 1, b);
        }
    }

    while(!feof(a)){ // resto de A
        fwrite(&ea, sizeof(Endereco), 1, saida);
        fread(&ea, sizeof(Endereco), 1, a);
    }
    while(!feof(b)){ // resto de B
        fwrite(&eb, sizeof(Endereco), 1, saida);
        fread(&eb, sizeof(Endereco), 1, b);
    }
    fclose(a);
    fclose(b);
    fclose(saida);
}
```

ERRADO: while(!feof(f)){ fread; fwrite; } → testa antes de ler → duplica o último registro

8.4 Chamadas (PARTES – 1 vezes)

```
int arquivoNome = PARTES;
for(int i = 0; i < (PARTES - 1) * 2; i += 2){
    char nome1[20];
    char nome2[20];
    char nome3[20];
    sprintf(nome1, "cep %d.dat", i);
    sprintf(nome2, "cep %d.dat", i + 1);
    sprintf(nome3, "cep %d.dat", arquivoNome); // ou PARTES + i/2
    intercala(nome1, nome2, nome3);
    arquivoNome++;
}
free(e);
```

i=0 cep_0 +cep_1 → cep_8 i=8 cep_8 +cep_9 → cep_12
i=2 cep_2 +cep_3 → cep_9 i=10 cep_10+cep_11 → cep_13
i=4 cep_4 +cep_5 → cep_10 i=12 cep_12+cep_13 → cep_14 *
i=6 cep_6 +cep_7 → cep_11
i += 2 → 2 arquivos/volta · (8-1)*2 = 14 → i = 0..12 (7 voltas)

quantidadeIntercala = PARTES – 1 → usar quantidadeIntercala * 2. **Não subtrair o 1 duas vezes!**

Ideia central

Registros têm tamanho fixo → BYTES ↔ REGISTROS

qtd de registros = tamanho do arquivo ÷ sizeof(Endereco)
posição em bytes = nº do registro × sizeof(Endereco)
300.000 ÷ 300 = 1.000 reg · 499 × 300 = byte 149.700

tamanho fixo

- calcular posição → fseek → acesso direto
 - busca binária (fseek a cada passo)
 - índice (fseek só no encontrado)
- contar e dividir registros → ordenação externa (sem fseek)

arquivo grande → conta (tamanho ÷ sizeof) → divide (divisao + resto)
→ qsort em cada parte → intercala 2 a 2 (PARTES – 1) → final ordenado

Lembretes de uma linha

PONTEIROS	f → endereço do FILE · *f → FILE · &e → endereço *e → conteúdo · . → struct · -> → ponteiro p/ struct
ARQUIVO	fseek MOVE (ONDE) · fread LÊ E AVANÇA (O QUE) ftell INFORMA · rewind VOLTA AO INÍCIO SEEK SET início · SEEK CUR atual · SEEK END fim fgetc 1 byte · fread itens · int c (EOF) ler → testar → usar → ler de novo rewind depois do SEEK END (se for dar fread)
CONTAS	BYTES ÷ sizeof → REG · REG × sizeof → BYTES
MEMÓRIA	malloc reserva · memset preenche · free libera qsort: QTD antes do TAMANHO (fread é o contrário)
PROGRAMAS	busca direta → arquivo ordenado + fseek a cada tentativa índice → chave+posição → ordena → busca → fseek externa → divide → qsort → intercala 2 a 2 → 1 arquivo PARTES 8 → 7 intercalações → final cep_14 (2×PARTES-2) malloc((divisao + 1) * sizeof(Endereco))